ESGI · ASSEMBLEUR X86

Fiche — Le débogueur gdb, commande par commande

Ce que l'on tape, ce que la machine répond — TP 1 et TP 2

Enseignant : **Youssef YOUNES**

Année universitaire 2026-2027

Polys générés à l'aide d'une IA — contenu relu et validé par l'enseignant.

À RETENIR
`make trace` fait tout seul ce que vous pouvez faire à la main avec `gdb` : exécuter une instruction, regarder les registres, recommencer. Cette fiche rassemble tout ce qu'il faut pour le faire soi-même. Toutes les sorties ont été tapées telles quelles sur l'image Docker du cours ; seules les adresses (les grands nombres en `0x8...` ou `0x4...`) peuvent changer d'une machine à l'autre.

1. Lancer et lire l'écran

Dans le dossier d'un exercice :

```
make debug                                  # s'arrête sur la première fonction du
fichier
make debug BREAK=somme                      # s'arrête à l'entrée de somme
make debug BREAK=maximum IN=tests/03-a-negatif.in    # et choisit l'entrée
```

Cela compile, ouvre `gdb`, pose un point d'arrêt à l'entrée de la fonction, lance le programme avec le fichier d'entrée et le fige à cet endroit. L'écran se divise en trois zones :

En haut	**Les registres (`eax`, `ecx`, `edx`, …, `eip`, `eflags`). Un registre qui vient de changer s'affiche en surbrillance.**
Au milieu	Votre fichier `.asm`. La ligne surlignée est la prochaine instruction, celle qui n'a pas encore été exécutée.
En bas	La ligne de commande. Vous tapez une commande après `(gdb)` et validez avec Entrée.

```
Breakpoint 1, calcul () at observer.asm:16
16      mov  eax, 10        ; eax = 10
```

Lecture : le programme est arrêté avant la ligne 16, `eax` ne vaut pas encore 10. Trois réflexes :

- Entrée sur une ligne vide répète la dernière commande (pour enchaîner les `si`) ;
- Ctrl-L redessine l'écran s'il est brouillé par la sortie du programme ;
- ne tapez jamais `run` : le programme est déjà lancé. Sur Mac Apple Silicon, `run` échoue d'ailleurs toujours (le programme tourne dans `qemu`, `gdb` y est seulement connecté). Si vous l'avez fait : `q`, puis `make debug` à nouveau.

2. Une séance complète (exercice 00 du TP 1)

`si` (step instruction) exécute une instruction et affiche la suivante. `p` (print) affiche une valeur ; un registre se tape avec un `$` devant.

```
(gdb) si
17      mov  edx, 3         ; edx = 3
(gdb) p $eax
$1 = 10
(gdb) si
18      add  eax, edx       ; eax = eax + edx
(gdb) si
19      sub  eax, 5         ; eax = eax - 5
(gdb) p $eax
$3 = 13
(gdb) info registers eax ecx edx
eax            0xd                 13
ecx            0x40800d20          1082133792
edx            0x3                 3
```

Le `$1`, `$3` est un numéro que `gdb` donne à chaque valeur affichée ; ignorez-le. `info registers` affiche chaque registre en hexadécimal puis en décimal. `ecx` contient un grand nombre quelconque : personne n'y a encore écrit, un registre n'est jamais vide. Cinq `si` plus tard :

```
24      ret                 ; retour : main.c affiche eax
(gdb) p $eax
$4 = 44
(gdb) p/x $eax
$6 = 0x2c
(gdb) x/i $pc
=> 0x80496f9 <calcul+25>: ret
(gdb) finish
0x08049710 in main () at main.c:11
(gdb) p $eax
$7 = 44
(gdb) continue
calcul() = 44
[Inferior 1 (process 1) exited normally]
(gdb) q
```

> **À RETENIR**
> Après `finish`, `gdb` n'affiche pas la valeur retournée par une fonction écrite en assembleur (il ne connaît pas son type). Tapez `p $eax`.

3. Les commandes

Commande	Effet
Avancer	
`si`	Exécute une instruction (entre dans les `call`). Entrée seule répète.

Commande	Effet
`ni`	Exécute une instruction, mais passe par-dessus un `call` sans y entrer.
`continue` ou `c`	Exécute jusqu'au prochain point d'arrêt ou jusqu'à la fin.
`finish`	Termine la fonction courante et revient dans `main.c`. Puis `p $eax`.
`break fin`	Ajoute un point d'arrêt sur l'étiquette ou la fonction `fin`. `info breakpoints` les liste.
`Ctrl-C`	Interrompt un programme qui tourne sans fin (boucle infinie). Puis `p $ecx`.
Lire les registres	
`p $eax`	`eax` en décimal signé. `p/x $eax` en hexadécimal, `p/c $eax` comme caractère.
`p $dl`	Le registre 8 bits `dl` (l'octet bas de `edx`). `p/c $dl` le montre comme caractère.
`info registers eax ecx edx`	Plusieurs registres d'un coup (hexa + décimal). Sans argument : tous.
`info registers eflags`	Les drapeaux laissés par le dernier `cmp` : `ZF` (égal), `SF` (résultat négatif)…
`display $eax`	Affiche `eax` automatiquement après chaque `si`. `display/x $esi` en hexa. `undisplay` pour arrêter.
`x/i $pc`	L'instruction sur laquelle le processeur est arrêté (`$pc` est `eip`).
Lire la mémoire	
`p a`	La valeur de la variable `a` du fichier `.asm`. `p &a` : son adresse. `p tableau[1]` : une case.
`x/dw &a`	Un mot de 4 octets (`w`) en décimal (`d`) à l'adresse de `a`. `x/2dw &a` : `a` puis `b`.
`x/3dw $esi`	Trois entiers à partir de l'adresse contenue dans `esi`.
`x/8cb texte`	Huit octets (`b`) comme caractères (`c`) : `98 'b' 111 'o' ...`
`x/8xb texte`	Huit octets en hexadécimal. `x/8db` : en décimal.
`x/s texte`	La chaîne de caractères rangée à cette adresse, jusqu'au 0 final. `x/s $esi` : à partir de la position courante.
`x/wx $esp`	Le mot au sommet de la pile. Juste après un `call` : l'adresse de retour.
`p *(int*)$esi`	L'entier de 4 octets à l'adresse contenue dans `esi` (ce que l'assembleur note `[esi]`).
Écrire	
`set $eax = 10`	Écrit 10 dans `eax`, comme `mov eax, 10`. Voir la section 5.
`set var a = 7`	Écrit 7 dans la variable `a`. `set var tableau[1] = 20` pour une case.
`set $pc = fin_max`	Fait repartir l'exécution à l'étiquette `fin_max`, comme un saut.

Commande	Effet
Divers	
`Ctrl-L`	Redessine l'écran.
`q`	Quitte `gdb`.

4. Les formats de `x`

`x` (examine) se lit `x/` puis combien, comment, quelle taille, puis l'adresse.

Comment	Taille	Exemples
`d` décimal signé, `x` hexadécimal, `c` caractère, `s` chaîne, `i` instruction	`b` octet (1), `w` mot (4)	`x/3dw $esi`, `x/8cb texte`, `x/s texte`, `x/i $pc`, `x/2xb $esi`

L'adresse peut être une variable (`texte` , `&a`), un registre (`$esi` , `$esp` , `$pc`) ou un nombre (`0x810b048`). Trois façons de voir le même mot `bonjour` :

```
(gdb) x/s texte
0x810b048:  "bonjour"
(gdb) x/8cb texte
0x810b048:  98 'b'  111 'o' 110 'n' 106 'j' 111 'o' 117 'u' 114 'r' 0 '\000'
(gdb) x/8xb texte
0x810b048:  0x62  0x6f  0x6e  0x6a  0x6f  0x75  0x72  0x00
```

5. Chaque instruction et son équivalent dans gdb

On ne peut pas taper `mov eax, 10` après `(gdb)` : `gdb` n'est pas un assembleur et répond `No symbol "mov" in current context.` Les instructions s'écrivent dans le fichier `.asm`. En revanche, `gdb` sait faire la même chose qu'une instruction avec `set` : le nom d'un registre prend un `$`, le nom d'une variable n'en prend pas, et le signe `=` joue le rôle de la virgule (destination à gauche, source à droite).

Instruction dans le `.asm`	Même effet, tapé dans `gdb`	Vérification
Copier et calculer		
`mov eax, 10`	`set $eax = 10`	`p $eax` donne `10`
`mov ecx, eax`	`set $ecx = $eax`	`p $ecx`
`add eax, edx`	`set $eax = $eax + $edx`	`p $eax`
`sub eax, 5`	`set $eax = $eax - 5`	`p $eax`
`imul eax, edx`	`set $eax = $eax * $edx`	`p $eax`
`neg eax`	`set $eax = -$eax`	`-3` devient `3`
`inc ecx` / `dec ecx`	`set $ecx = $ecx + 1` / `- 1`	`p $ecx`
Lire et écrire la mémoire		

Instruction dans le `.asm`	Même effet, tapé dans `gdb`	Vérification
`mov eax, [a]`	`set $eax = a`	`p $eax` donne la valeur de `a`
`mov esi, tableau`	`set $esi = &tableau`	`p/x $esi` donne l'adresse
`add eax, [esi]`	`set $eax = $eax + *(int*)$esi`	`p *(int*)$esi` : la case pointée
`add esi, 4`	`set $esi = $esi + 4`	`p/x $esi` a augmenté de 4
`movzx eax, byte [texte]`	`set $eax = texte[0]`	`p/c $eax` donne `98 'b'`
`mov dl, [c]`	`set $dl = c`	`p/c $dl`
`cmp byte [esi], 0`	`p *(char*)$esi == 0`	`1` : fin de chaîne
`sub byte [esi], 32`	`set var *(char*)$esi = *(char*)$esi - 32`	`x/s texte`
`inc esi` (un octet)	`set $esi = $esi + 1`	`x/s $esi`
écrire `7` dans la variable `a`	`set var a = 7`	`x/dw &a` donne `7`
Comparer et sauter		
`cmp eax, edx` puis `jge ...`	`p $eax >= $edx`	`1` : le saut aurait lieu, `0` : non
`cmp eax, edx` puis `jl ...`	`p $eax < $edx`	idem
`cmp ecx, 0` puis `je ...`	`p $ecx == 0`	idem
`jmp boucle` (ou un saut pris)	`set $pc = boucle`	`x/i $pc` montre l'instruction de `boucle`
Appels et pile		
`call carre`	`si` entre dedans ; `ni` l'exécute en entier	`x/wx $esp` : l'adresse de retour
`ret`	`finish`	puis `p $eax`
`push ebx` / `pop ebx`	`si`, puis `p/x $esp` (−4 / +4)	`x/wx $esp` après le `push`
Division et appel système		
`cdq` puis `idiv dword [b]`	`set $edx = $eax % b` puis `set $eax = $eax / b`	`17 / 5` donne `3`, reste `2`
`int 0x80` (write)	`si` : le texte apparaît	`p $eax` : le nombre d'octets écrits

Séance type où l'on rejoue `calcul` de l'exercice 00 à la main, sans exécuter une seule instruction du programme :

```
(gdb) set $eax = 10
(gdb) set $edx = 3
(gdb) set $eax = $eax + $edx
(gdb) set $eax = $eax - 5
(gdb) set $eax = $eax * $edx
(gdb) set $ecx = $eax
```

```
(gdb) set $ecx = $ecx - 4
(gdb) set $eax = $eax + $ecx
(gdb) p $eax
$1 = 44
```

Remarques : ce que `set` met dans un registre est écrasé dès que le programme exécute une instruction qui écrit dedans (`set $eax = 100` puis `si` sur `mov eax, 10` redonne 10). `cmp` ne change aucun registre : on pose directement la question au processeur avec `p $eax >= $edx`. Un saut ne calcule rien, il change `eip` : `set $pc = fin_max` fait ce que ferait `jge fin_max` quand la condition est vraie. `idiv` produit le reste dans `edx` et le quotient dans `eax` : écrivez le reste d'abord, car le quotient écrase `eax` ; `p -17 / 5` donne `-3` et `p -17 % 5` donne `-2`, comme le processeur. `p` sert aussi de calculatrice : `p 10 + 3`, `p/x 255`, `p/c 65`.

6. Les pièges

Le programme se termine sans s'arrêter. Au TP 1 (ex 01 à 04), `make debug` sans `BREAK=` affiche `Function "a" not defined.` puis la sortie du programme : le point d'arrêt par défaut est posé sur le premier nom `global` du fichier, qui est une variable. Donnez le nom de la fonction : `make debug BREAK=somme`. Au TP 2, les fichiers déclarent la fonction en premier, `make debug` tout court suffit.

`si` sur le `ret`. Vous retombez dans `main.c`, du C compilé. `finish`, `continue` ou `q`.

Le programme attend au clavier. `make debug` branche le premier fichier `tests/*.in` sur l'entrée : vous n'avez rien à taper. Si rien ne se passe, vous avez tapé `run`. `q`, puis `make debug`.

Boucle infinie. `break fin` puis `continue` ne revient jamais. `Ctrl-C` interrompt, `p $ecx` montre le compteur qui ne bouge pas, `x/i $pc` montre où l'on tourne.

`Cannot access memory at address 0x1`. `x/dw $esi` avec un `esi` qui contient une valeur au lieu d'une adresse (`mov esi, [tableau]` avec crochets en trop). C'est la `Segmentation fault` expliquée.

Sur Mac Apple Silicon, le `si` sur `int 0x80` exécute aussi l'instruction suivante : particularité de l'émulateur.